arXiv:2506.02943v1 [cs.SE] 3 Jun 2025

A Multi-agent LLM-based JUit Test Generation with Strong Oracles

Qinghua Xu
Research Ireland Lero Centre
University of Limerick
Limerick, Ireland
qinghua.xu@ul.ie

Guancheng Wang
Research Ireland Lero Centre
University of Limerick
Limerick, Ireland
guancheng.wang@ul.ie

Lionel C. Briand
Research Ireland Lero Centre
University of Limerick, Ireland
University of Ottawa, Canada
lionel.briand@lero.ie

Kui Liu
Software Engineering Application Technology Lab
Huawei
Hangzhou, China
brucekuiliu@gmail.com

June 4, 2025

Abstract

Unit testing plays a critical role in ensuring software correctness. However, writing unit tests manually is labor-intensive, especially for strong typed languages like Java, motivating the need for automated approaches. Traditional methods primarily rely on search-based or randomized algorithms to generate tests that achieve high code coverage and produce regression oracles, which are assertions derived from the program's current behavior rather than its intended functionality. Recent advances in large language models (LLMs) have enabled oracle generation from natural language descriptions, aligning better with user requirements. However, existing LLM-based methods often require LLM fine-tuning or rely on external tools such as EvoSuite for test prefix generation.

In this work, we propose CANDOR, a novel end-to-end, prompt engineering-based LLM framework for automated unit test generation in Java. CANDOR orchestrates multiple specialized LLM agents to collaboratively generate complete JUnit tests, including both high-quality test prefixes and accurate oracles. To mitigate the notorious hallucinations in LLMs and improve oracle correctness, we introduce a novel strategy that engages multiple reasoning LLMs in a panel discussion and generate accurate oracles based on consensus. Additionally, to reduce the verbosity of reasoning LLMs' outputs, we propose a novel dual-LLM pipeline to produce concise and structured oracle evaluations.

Our experiments on the HumanEvalJava and LeetCodeJava datasets show that CANDOR can generate accurate oracles and is slightly better than EvoSuite in generating tests with high line coverage and clearly superior in terms of mutation score. Moreover, CANDOR significantly outperforms the state-of-the-art, prompt-based test generator LLM-Empirical, achieving improvements of 15.8 to 25.1 percentage points in oracle correctness on both correct and faulty source code. Further ablation studies confirm the critical contributions of key agents in improving test prefix quality and oracle accuracy.

Keywords Automated Testing, Unit Test Generation, Large Language Models

1 Introduction

Unit testing is a software testing activity wherein individual units of code are tested in isolation Siddiq et al. [2024]. Unit testing plays a crucial role in modern software development because it helps software developers identify and fix defects in early phases Beck [2022]. However, manually creating unit tests is laborious and necessitates substantial

domain expertise to craft high-quality tests Daka et al. [2015], Daka and Fraser [2014]. Consequently, developers often forgo writing tests for their code altogether. As reported in Siddiq et al. [2024], Gonzalez et al. [2017], only 17 % of 82,447 studied Github projects contained test files.

To reduce the burden of manual test writing, various solutions have been proposed to automatically generate test suites Fraser and Arcuri [2011a], Pacheco et al. [2007], Shamshiri et al. [2015]. A test suite typically comprises a set of test cases, where each test case is composed of a *test prefix* — inputs to the system under test (SUT) — and a *test oracle* that verifies whether the actual behavior matches the expected behavior of the SUT Hossain and Dwyer [2024]. Prior works in automated test generation predominantly focuses on test prefix generation, with a goal of maximizing code coverage Tullis and Benjamin [2011], Serra et al. [2019]. Techniques in this area include fuzzing Miller et al. [1990], Fioraldi et al. [2023], feedback-directed random test generation Csallner and Smaragdakis [2004], Pacheco and Ernst [2007], Pacheco et al. [2008], Selakovic et al. [2018], Arteca et al. [2022], dynamic symbolic execution Godefroid et al. [2005], Sen et al. [2005], Cadar et al. [2008], Tillmann et al. [2014], and search/evolutionary algorithm-based approaches Fraser and Arcuri [2011b,a]. Although these techniques are successful in generating test suites with high code coverage, they are still not frequently employed in practice Fraser et al. [2015]. Most of them rely on static or dynamic analysis to explore control and data flow paths in the program and consequently generate tests that maximize code coverage. One major limitation lies in their reliance on oracles that are automatically derived from the system's current behavior. These are typical *regression oracles*, which assume the existing implementation is correct and simply record its behavior as the expected outcome. As such, they are primarily useful for detecting behavioral changes across software versions, not for validating functional correctness against intended specifications.

Recent advances in machine learning, especially the emergence of Large Language Models (LLMs), have inspired several works in generating test suites with specification-based oracles Hossain and Dwyer [2024], Dinella et al. [2022], Molina et al. [2024]. Empirical studies have shown that LLMs can achieve state-of-the-art (SOTA) performance in oracle generation for weakly typed languages like JavaScript. For instance, a recent study Schäfer et al. [2023] found that LLMs can generate correct test oracles for JavaScript functions with high accuracy. However, test oracle generation for strongly typed languages such as Java remains significantly more challenging. This is due to the need for strict adherence to type constraints, language-specific syntax, and more complex execution semantics. To address this, recent work such as TOGLL Hossain and Dwyer [2024] combines EvoSuite-generated Fraser and Arcuri [2011a] test prefixes with a fine-tuned LLM for oracle generation. While effective, this hybrid approach has several limitations. First, it relies on EvoSuite to produce initial test scaffolds, which inherits the known limitations of EvoSuite, such as brittle test structures and inadequate test diversity Lemieux et al. [2023]. Second, it requires fine-tuning on Java test data, which is costly to collect, often unavailable, and may not generalize well to new projects or domains.

In contrast, we advocate for a purely prompt-based approach using off-the-shelf LLMs, which is more practical, and widely accessible. Siddiq et al. Siddiq et al. [2024] explored this direction by using LLMs to perform end-to-end test generation for Java. Their method achieved lower line coverage on the HumanEvalJava benchmark than EvoSuite, and suffered from low oracle correctness. This is largely due to the uncertainty and hallucination of LLMs, which often produce assertions that appear plausible but are semantically incorrect.

To tackle these challenges, we propose a novel approach, CANDOR, for automated JUnit test generation with accurate oracles using off-the-shelf LLMs. CANDOR is an end-to-end test generation framework that eliminates the dependency on EvoSuite for test prefix generation, relies entirely on off-the-shelf LLMs without fine-tuning.

For test prefix generation, CANDOR coordinates multiple specialized LLM agents—*Initializer*, *Planner*, *Tester*, and *Inspector*—to iteratively construct and refine test files. These agents are responsible for generating an initial test scaffold, designing test plans to improve coverage, producing executable test cases, and inspecting test case quality, respectively. At this stage, oracles are generated directly from the source code, which may be faulty in practice, and thus the resulting assertions—referred to as tentative oracles—may be incorrect.

To ensure oracle correctness, CANDOR introduces a novel ensemble-based strategy inspired by David Hume's quote, "*Truth springs from arguments amongst friends,*", which underscores that constructive dialogue and diverse perspective lead to robust conclusions. In this spirit, CANDOR employs a panel discussion-style approach where multiple *Panelist* agents, powered by reasoning LLMs such as DeepSeek R1 Guo et al. [2025], independently evaluate tentative oracles against requirements derived from the natural language description of the SUT. Despite their effectiveness, such LLMs often suffer from verbose and redundant outputs, known as the "overthinking phenomenon" Sui et al. [2025]. To address this, CANDOR introduce a dual-LLM pipeline, where a basic LLM (i.e., without reasoning capability) agent *Interpreter* extracts and formats key insights from each *Panelist*'s output. Finally, a *Curator* agent, also a basic LLM, aggregates these interpretations from the pipelines, determines necessary corrections, and generates accurate oracles through consensus. This design mitigates hallucination and uncertainty in test oracles by enabling multiple LLM agents to cross-validate tentative oracles and reach a consensus, ensuring that the final oracles align more closely with the intended behavior described in the specification.

We evaluate CANDOR in terms of the quality of both test prefix and oracles on the benchmarking dataset HumanEval-Java Athiwaratkun et al. [2022] and a Leetcode-derived dataset from a popular Github repository 202 [Accessed: 2025a] that includes code with two complexity levels. Experimental results show that CANDOR is slightly better than EvoSuite in generating high-coverage test prefixes and clearly better in terms of mutation score, while producing accurate specification-based oracles. CANDOR also outperforms the SOTA end-to-end, LLM-based method LLM-Empirical by at least 15.8 percentage points in terms of oracle correctness on both correct and faulty source code. We also assess the individual contribution of the key agents in CANDOR. Experimental results show that removing *Planner* leads to substantial decrease in line coverage (0.099), branch coverage (0.130) and mutation score (0.111), respectively. Removing *Requirement Engineer*, and the panel discussion decreases oracle correctness by at least 0.005 and 0.158, respectively.

In summary, our contributions are as follows.

1. To the best of our knowledge, CANDOR is the first multi-agent LLM framework for end-to-end JUnit test generation, where specialized agents collaborate to iteratively generate, validate, and refine test cases, and do so without relying on external tools like EvoSuite or LLM fine-tuning.
2. We introduce a novel panel discussion-inspired strategy for test oracle generation, mitigating LLMs' hallucination and uncertainty.
3. To address the "overthinking phenomenon" of reasoning LLMs, we design a novel dual-LLM pipeline to extract concise oracle evaluations from their verbose output.
4. Experimental results on HumanEvalJava and LeetCode demonstrate that CANDOR compares favorably with EvoSuite in generating high quality test prefixes with slightly better line coverage and clearly higher mutation scores, while producing specification-based oracles with high accuracy, substantially outperforming a SOTA baseline by at least 15.8 percentage points.

2 Running Example

In this section, we present the running example in Figure 1, which is adapted from the HumanEvalJava dataset Chen et al. [2021]. For each subject under test (SUT), we assume the availability of two relevant pieces of information: the *source code* and its natural language *description*.

Description. This part provides a natural language summary of the SUT's main functionality. We assume such a description is available, as programmers typically provide such documentation explaining their code Siddiq et al. [2024]. This description should include the input, the core implementation logic and the expected output. As depicted in Figure 1, the *description* specifies the SUT should take a list of integers as input, implement element-wise transformation and return the sum of the transformed list as output. For the ith element $lst[i]$, the transformation rule is as in Equation 1.

$$lst[i] = \begin{cases} lst[i]^2, & \text{if } (i\%3) == 0 \\ lst[i]^3, & \text{if } (i\%3)! = 0 \text{ and } (i\%4) == 0 \\ lst[i], & \text{otherwise} \end{cases} \tag{1}$$

Source Code. The *source code* is an implementation of the logic described in the *description*, using the specified input and producing the expected output. For example, as shown in the second box of Figure 1, it defines a class "SumSquares1" with a static method "sumSquares". In this example, the implementation is faulty: the condition "$i\%3! = 0$" from Equation 1 is incorrectly written as "$i\%3 == 0$".

This example represents a realistic and challenging test generation scenario, where the SUT is faulty and thus unreliable for oracle derivation. In real-world development, especially during early-stage or iterative coding, it is common for source code to contain latent bugs or incomplete logic Ettles et al. [2018]. The goal of this paper is to generate high quality tests with accurate oracles, by combining both the *source code* and its natural language *description*, thus ensuring that the generated oracles are aligned with the intended functionality.

3 Methodology

This section demonstrates the workflow of CANDOR, comprising three steps: *Initialization*, *Test Prefix Generation* and *Oracle Fixing*. In the *Initialization* step, an initial test file v_0 is generated from the source code and iteratively refined by fixing syntactic errors found by a validation process, resulting in a syntactically correct test file v_1. This file, typically containing only a small number of test cases (often ≤ 3), serves as a template for the subsequent steps since it conforms to the syntax and conventions of Java and the testing framework JUnit 5. However, v_1 usually has low code coverage, as the focus at this stage is on syntactic correctness rather than test completeness. To enhance code coverage, the *Test*

```
/**
*
* This function will take a list of integers.
* For all entries in the list, the function shall square the integer entry
if its index is a multiple of 3 and will cube the integer entry if its
index is a multiple of 4 and not a multiple of 3.
* The function will not change the entries in the list whose indexes are
not a multiple of 3 or 4.
• The function shall then return the sum of all entries.
*/
```

Description Example

```
package original;
import java.util.ArrayList;
import java.util.List;


class SumSquares1 {
    public static int sumSquares(List<Object> lst) {
        List<Integer> result = new ArrayList<Integer>();
        for (int i = 0; i < lst.size(); i++) {
            if (i % 3 == 0) {
                result.add((int) lst.get(i) * (int) lst.get(i));
            }
            else if (i % 4 == 0 && i % 3 == 0) {
                result.add((int) lst.get(i) * (int) lst.get(i) * (int)
            lst.get(i));
            }
            else {
                result.add((int) lst.get(i));
            }
        }

        int sum = 0;
        for (int i = 0; i < result.size(); i++) {
            sum += result.get(i);
        }
        return sum;
    }
}
```

Source Code Example

Figure 1: Running example from the HumanEvalJava dataset

Prefix Generation step generates additional valid test prefixes with tentative oracles, producing an expanded test file v_2. Notably, both Steps I and II rely solely on the source code for test generation. As a result, if the source code is faulty, the tentative oracles in v_0, v_1, and v_2 may also be incorrect. To address this issue, the *Oracle Fixing* step conducts a panel discussion among multiple LLM agents to revise and correct the oracles in v_2, yielding the final test file v_f.

3.1 Step I: Initialization

This step aims to generate a syntactically correct initial test file. As shown in the first box of Figure 2, it involves two main components: an LLM agent called the *Initializer* and a *Validation* process. Together, they form an iterative loop to ensure the generation of a valid initial test file v_1.

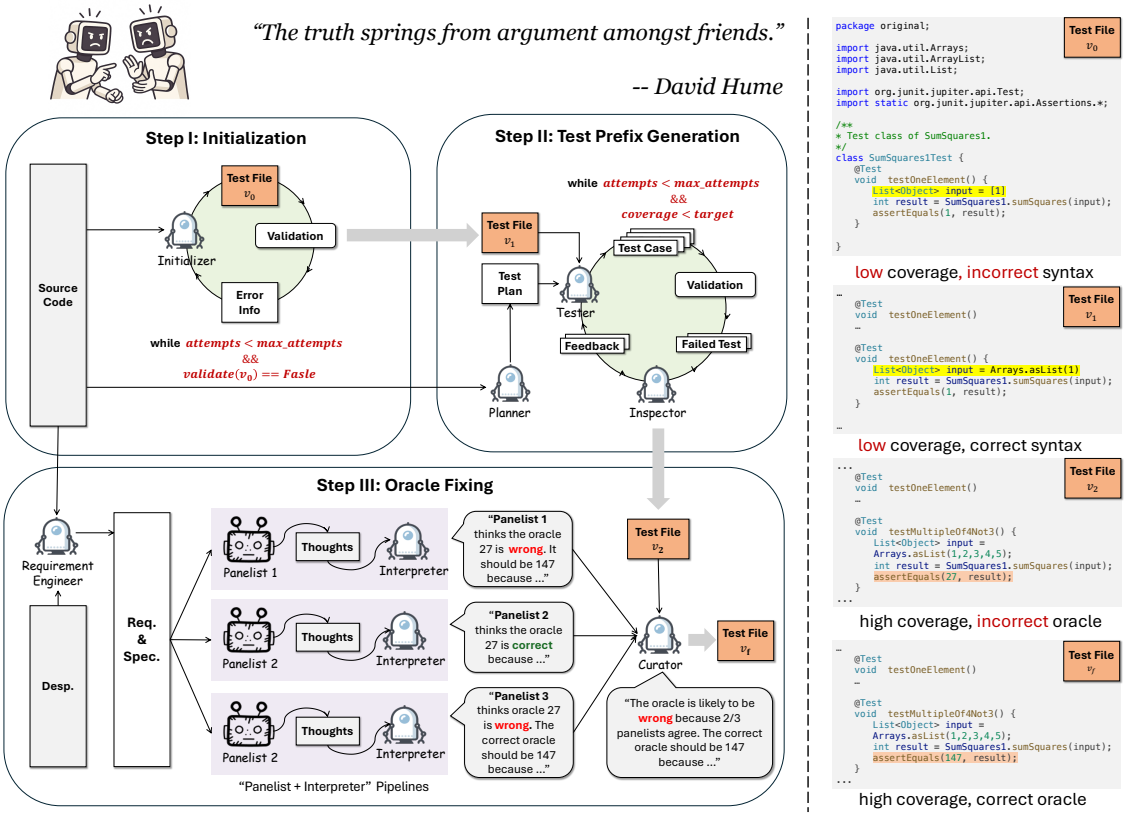

Figure 2: Overview of CANDOR and examples. "Desp.", "Req." and "Spec." are short for "Description", "Requirement" and "Specification", respectively.

Initializer. The *Initializer* is a basic LLM agent responsible for producing an initial test file v_0 based on the given source code. As illustrated in the first profile in Figure 3, the *Initializer* is guided by two types of prompts: a system prompt, which defines the model's role (e.g., as a software testing expert), and a user prompt, which provides specific inputs such as the source code, last failed test file, error messages, Java and JUnit 5 conventions and output format. The *Initializer* responds with a JSON object containing a single field, `test_file`, which holds the complete generated test code. Given the source file provided in Section 2, we present the generated initialize test file v_0 under Profile 1 of Figure 3, including one test case `testOneElement`.

Validation. The generated test file v_0 is then passed to the *Validation* component, which attempts to compile and execute it. At this stage, v_0 often contains syntactic errors and achieves low code coverage. In the example of v_0 showed in Figure 2, the input list variable is incorrectly instantiated as `List<Object> input = [1]` following Python instead of Java conventions. If syntax-related errors are detected, the error messages are fed back to the *Initializer* for refinement. This process is repeated until a syntactically correct and error-free test file v_1 is produced or a maximum number of attempts (`max_attempts`) is reached. An example of v_1 is shown in Figure 2, where the input list variable is correctly instantiated as `List<Object> input = ArrayList.asList(1)` following Java conventions. While v_1 is syntactically valid and adheres to language and testing framework conventions, it typically still has low coverage, relying on subsequent steps for coverage improvement.

3.2 Step II: Test Prefix Generation

This step takes the initial test file v_1 from Step I and aims to enhance code coverage by generating additional test cases. As shown in the second box of Figure 2, this step involves the *Validation* process (same as in Step I) and three LLM agents: *Planner*, *Tester*, and *Inspector*. These agents collaborate in an iterative loop to propose, generate, and refine new test cases.

Figure 3: Profiles of LLM agents. "Basic LLM" and "Reasoning LLM" refers to LLMs without and with reasoning capability, respectively. Each profile consists of a system prompt and a user prompt, which define the agent's role and task, respectively. Fields enclosed in double curly braces (e.g., `{{...}}`) in the user prompt represent variables that are dynamically filled for each specific case. The field `format_instructions` is included in all user prompts and specifies the expected output format for the agent. Below each profile, an example output is provided based on the running example.

Planner. The *Planner* is a basic LLM agent responsible for analyzing coverage gaps and proposing new test cases accordingly. As shown in the second profile in Figure 3, the system prompt defines its role as a test plan designer tasked with improving code coverage. The user prompt requires inputs including the source code, uncovered lines from the Jacoco coverage report, and the current test file. Based on this information, the Planner outputs a list of structured test plans in JSON format under the field `test_cases_to_add`, with each test case described by a `name`, a brief natural language `description`, `input`, and `expected output`. A snippet of a generated plan based on the running example is provided in Profile 2, Figure 3. This plan proposes two test cases to enhance code coverage: `Test empty list` and `Test list with multiple of 4 not 3 index`.

Tester. The *Tester* is a basic LLM agent that generates executable test code based on the plans proposed by the *Planner*. As shown in the third profile in Figure 3, the system prompt defines the *Tester*'s role as a Java unit test generator following JUnit 5 and the conventions of the given initial test file. The user prompt includes the source code, the test plan from the *Planner*, and feedback from the *Inspector* (if any). The output is a JSON object containing a list of `generated_test_cases`, with each entry specifying the `behavior`—a natural language explanation of what the test is verifying, `test_name`, `test_code`, and any `new_import_statements` required to execute this test case. A snippet of the generated tests based on the running example is provided under Profile 3 in Figure 3. The *Tester* generates the test case `testMultipleOf4Not3`, with a test prefix of `ArrayList.asList(1,2,3,4,5)` and a tentative oracle `assertEquals(27, SumSquares1.sumSquares(input))`. Notice the tentative oracle is incorrect: it is calculated as $1 + 2 + 3 + 4^2 + 5 = 27$, whereas the correct oracle should be $1 + 2 + 3 + 4^2 + 5^3 = 147$. The fifth element was not cubed as specified in the *description* due to a bug in the *source code*, specifically the conditional statement `if (i%4==0&&i%3==0)` (Figure 1). Additionally, the *Tester* fails to identify the necessary `import` statements for `ArrayList`.

Inspector. The *Inspector* is a basic LLM agent that evaluates the compiled and executed test cases to detect and explain any errors. Its role, as defined in the fourth profile in Figure 3, is to inspect the generated test code and offer feedback for improvements. The user prompt includes the source code and error messages from the validation process. The output consists of structured feedback entries, each containing the `failed_test_code`, the `error_message`, the `error_type`, and a `potential_fix`. This feedback is then sent back to the *Tester* for refinement. A snippet of the generated feedback based on the running example is presented under Profile 4 of Figure 3. In this case, the *Inspector* correctly identifies the missing `import` statements for `ArrayList` in the test file v_1. However, it does not flag the incorrect oracle, as the analysis is performed against the faulty *source code*, which exhibits the same buggy behavior as the test output.

Step II is iterative; the *Planner* suggests additional test cases, the *Tester* generates code, and the *Inspector* provides corrective feedback. The loop terminates either when the maximum number of attempts is reached or when the code coverage satisfies a predefined threshold. This step produces the test file v_2, which tends to have significantly higher coverage compared to v_0 and v_1, while maintaining syntactic correctness.

3.3 Step III: Oracle Fixing

The first two steps generate test cases and tentative oracles directly from the source code. However, when the source code is faulty, the tentative oracles may also be incorrect. To address this limitation, Step III focuses on fixing the oracles using natural language *descriptions*, which better reflect the intended functionality. As shown in Figure 2, this step involves a *Requirement Engineer* agent, multiple dual-LLM pipelines (*Panelist* + *Interpreter*), and a *Curator* agent. These agents collaborate to produce more accurate test oracles, even in the presence of incorrect code implementation.

Requirement Engineer The *Requirement Engineer* is a basic LLM agent responsible for extracting both natural language requirements and formal specifications from the provided description of the function. As illustrated in the fifth profile in Figure 3, the system prompt defines its role as a software engineer specializing in requirement analysis. The user prompt includes the textual description of the code, and the output consists of a set of human-readable requirements (e.g., input type and expected behavior). When possible, it also produces a formal specification expressed in predicate logic. If a formalization is not applicable or too ambiguous for the task, the specification is left empty, and only the natural language requirements are used for downstream oracle correction. The extracted requirements and specifications for the running example are shown under Profile 5 in Figure 3.

Dual-LLM pipelines Each pipeline comprises a reasoning LLM agent *Panelist* and a basic LLM agent *Interpreter*. Each *Panelist* functions as an independent oracle corrector. As shown in the sixth profile in Figure 3, the system prompt defines its role as a testing expert tasked with identifying and fixing incorrect test oracles generated by automated tools. The user prompt requires the natural language description, the extracted requirements and specifications from

the *Requirement Engineer*, and the set of test cases generated in Step II. Each agent analyzes the given test case and determines whether the existing oracle is correct. If not, it provides a corrected oracle based on the intended behavior described in the requirements and specification. To mitigate the "overthinking phenomenon" (see Section 1) of reasoning LLMs, each panelist is paired with a basic LLM agent *Interpreter*. Each *Interpreter* is responsible for extracting relevant insights from *Panelist*'s verbose thoughts and produce structured evaluations of the tentative oracles. As shown in the seventh profile of Figure 3, the system prompt defines its role as an assistant working with an excellent but self-doubting tester. The user prompt requires the *Panelist*'s thoughts and the test code under examination. Example output of both *Panelist* and *Interpreter* based on the running example are shown under Profile 6 and Profile 7 of Figure 3, respectively. In this case, the *Panelist* generates a lengthy, self-questioning reasoning process — using phrases like `Wait` and `However` — while the *Interpreter* successfully extracts concise and structured oracle evaluations.

Curator Multiple dual-LLM pipelines operate in parallel, each making independent judgments of the tentative oracle. The *Curator* aggregates these judgments and determines the final oracle. As shown in the eighth profile of Figure 3, the system prompt defines its role as a leader of a software engineering team. While teams members are discussing the correctness of the tentative oracle, the *Curator* is responsible for summarizing and making a final judgment. Its user prompt requires individual judgment from each pipeline, and the test code under examination. An example output of *Curator* based on the running example is shown below Profile 8 of Figure 3. In this case, the *Curator* successfully identifies the tentative oracle as incorrect and revises it into the correct oracle `assertEquals(147, SumSquares1.sumSauares(input));`.

Step III yields the final test file v_f, which not only achieves high code coverage—like v_2—but also contains corrected oracles based on the natural language *description*.

4 Experimental Setup

To assess CANDOR, we ask the following research questions (RQs):

RQ.1 How well does CANDOR perform at generating high coverage test prefixes? How does its performance vary under different levels of code complexity?

RQ.2 Can CANDOR produce accurate test oracles, both when the source code is correct (i.e., conforms to the intended requirements) and when it is faulty?

RQ.3 What is the individual effectiveness contribution of key components in CANDOR, including the *Planner*, *Requirement Engineer* and the panel discussion?

RQ1 evaluates the overall effectiveness of the generated test cases by comparing code coverage and mutation scores with state-of-the-art baselines. We conduct experiments on the benchmark HumanEvalJava dataset Athiwaratkun et al. [2022]. Additionally, we evaluate CANDOR on a LeetCode-derived dataset (LeetCodeJava) that addresses problems of varying difficulty to assess its robustness across different levels of code complexity. For RQ2, we focus on the accuracy of oracle generation. We evaluate oracle correctness under two conditions: (1) when the source code is correct and conforms to the intended requirements, and (2) when the code is faulty. To simulate faulty scenarios, we adopt a commonly-used tool (PiTest 202 [Accessed: 2025b]) to inject mutations into the source code from both HumanEvalJava and LeetCodeJava. This evaluation investigates CANDOR's robustness and its practical utility in real-world testing scenarios. With RQ3, we assess the individual contributions of key components within CANDOR through a series of ablation studies. These studies focus on the *Planner*, *Requirement Engineer*, and the panel discussion. We do not perform ablation on other agents, as they are essential for generating valid test cases; removing them would render CANDOR non-functional.

4.1 Datasets

In this work, we focus on unit test generation for Java. We evaluate CANDOR on two datasets: HumanEvalJava Athiwaratkun et al. [2022] and LeetCodeJava 202 [Accessed: 2025a]. We exclude other datasets used in the literature such as Defects4J Just et al. [2014], SF110 Fraser and Arcuri [2011a], and OracleEval25 Hossain and Dwyer [2024], as they are project-level datasets that involve complex dependencies and user-defined classes. Incorporating such datasets requires additional handling of project-specific configurations, which is beyond the scope of this work. Moreover, these datasets do not consistently provide natural language *descriptions* of SUT—an essential component for our method, which relies on such *descriptions* to enable requirement extraction and oracle generation. We leave the integration and evaluation on project-level datasets for future work.

HumanEvalJava It consists of 160 Java programs implemented by Siddiq et al. to solve problems introduced in Athiwaratkun et al. [2022]. Each program defines a Java class under test that includes a single method. The length of these programs ranges from 20 to 222 lines, with an average of 41 lines of code.

LeetCodeJava It includes solutions to 3,000 programming problems, categorized by difficulty levels: "easy", "medium", and "hard". More difficult problems generally require more complex code. Due to the constraints of time and available computational resources, we randomly sampled 50 programs from "medium", and "hard" difficulty category, resulting in a total of 100 programs. These programs have an average of 33 lines of code, with a minimum of 7 and a maximum of 126. The average lines of code for "medium", and "hard" categories are 31, and 39, respectively.

4.2 Baselines

We compare CANDOR against two representative baselines: EvoSuite Fraser and Arcuri [2011a] and LLM-Empirical Siddiq et al. [2024].

EvoSuite This is a widely adopted automated test generation tool for Java programs that uses search-based techniques, such as genetic algorithms, to systematically generate unit tests. It analyzes the program under test to produce test cases aiming to maximize code coverage metrics like line coverage. In addition to generating test prefixes, EvoSuite automatically produces regression oracles—assertions that capture the current program behavior by checking expected outputs, exceptions, and state changes. These generated tests are structured as JUnit test cases, making EvoSuite a popular baseline for research in automated testing and regression testing Fraser and Arcuri [2011a], Hossain and Dwyer [2024], Tufano et al. [2020].

LLM-Empirical The LLM-Empirical method, proposed by Siddiq et al. Siddiq et al. [2024], leverages LLMs such as Codex, GPT-3.5-Turbo, and StarCoder to generate JUnit tests for Java programs using prompt engineering without fine-tuning. Unlike EvoSuite, which generates regression oracles based on the current implementation behavior, LLM-Empirical generates test oracles based on natural language descriptions of the SUT. It represents the SOTA in prompt-based, LLM-driven oracle generation for Java unit tests. Among the models used in LLM-Empirical, Codex (specifically, code-davinci-002) and GPT-3.5-Turbo are the most capable, but both are closed-source. Moreover, code-davinci-002 is no longer publicly available, and GPT-3.5-Turbo requires paid API access. As a result, for HumanEvalJava, we report the results from their original paper, and for LeetCodeJava, we report the results obtained using GPT-3.5-Turbo. Other advanced LLMs cannot be directly applied to LLM-Empirical because their implementation—particularly the component that fixes test cases—is specifically tailored to the behavior and outputs of the LLM they use.

We are also aware of TOGLL Hossain and Dwyer [2024], a recent oracle generation method proposed by Hossain et al. However, we exclude it from our evaluation for two main reasons: (1) it relies on EvoSuite-generated test prefixes and LLM fine-tuning, which differs from the objectives of our end-to-end, prompt-only approach; (2) it was published recently, and due to time and resource constraints, we were unable to implement it on our datasets. We leave evaluation with TOGLL for future work.

4.3 Evaluation Metrics and Statistical Testing

We evaluate the effectiveness of CANDOR for test generation in terms of code coverage, mutation score, and oracle correctness. To reduce the influence of randomness, we repeat all the experiments three times and calculate the average of each metric. For statistical testing, we compare these metrics between CANDOR and each baseline across programs using a paired non-parametric test to assess significance.

Code coverage This metric measures the proportion of code executed by passing tests in the generated test suites, typically reported as line coverage and branch coverage. Line coverage refers to the percentage of lines that are executed by the test suite, while branch coverage measures the percentage of control flow branches (e.g., if/else conditions) that are reached. Code coverage is a widely used metric for assessing the effectiveness of test prefixes, but it does not address test oracles Hemmati [2015].

Mutation score This calculates the percentage of synthetic bugs (mutants) detected by the test suite. We use PiTest with its standard mutation operators to generate mutants for each program under test. A mutant is considered killed if at least one test in the suite fails when executed against the mutated program. The mutation score is calculated as the percentage of killed mutants over the total number of generated mutants. A higher mutation score indicates a more fault-revealing test suite.

Oracle correctness This metric measures the accuracy of the assertions (oracles) in a generated test suite. Specifically, it is calculated as the number of correct oracles divided by the total number of oracles in the test suite. An oracle is considered correct if it accurately reflects the intended behavior of the program as specified by its natural language description or reference implementation.

Statistical testing To compare CANDOR against baselines, we perform the Wilcoxon Signed Rank test for all RQs, as recommended in Arcuri and Briand [2011], with a significance level of 0.05. The Wilcoxon Signed Rank test is a non-parametric statistical test used to determine whether there is a significant difference between two independent distributions (e.g., method A vs. method B). For RQ2, we also report the A_{12} effect size to measure the magnitude of difference in oracle correctness between CANDOR and LLM-Empirical. A_{12} ranges between 0 and 1, representing the probability that method A yields better results than method B.

4.4 Implementation

All experiments are conducted on a Precision 7960 Tower XCTO workstation equipped with an Intel Xeon w9-3495X processor and dual NVIDIA RTX 6000 Ada GPUs. The implementation is written in Python, using the LangChain library 202 [Accessed: 2025c] for LLM integration. We use LLama 3.1 70B as the basic LLM and DeepSeek R1 LLama-distilled 70B as the reasoning LLM. The max_attempt parameter in the Initialization and Test Prefix Generation step is set to be 3. We limit the number of output tokens of DeepSeek to 2000 for efficiency. The number of "Panelist + Interpreter" pipeline is set to be 3. To support reproducibility, we plan to release the code publicly upon paper acceptance.

5 Experiment Results

5.1 RQ1 Results: Test Prefix Quality

Table 1 presents the experimental results for RQ1, comparing CANDOR with LLM-Empirical and EvoSuite in terms of test prefix quality, measured by line coverage ("Line"), branch coverage ("Branch"), and mutation score ("Mutation"). Overall, CANDOR outperforms both baselines across all metrics, with one exception: on the Leetcode-Medium dataset, EvoSuite achieves slightly higher branch coverage than CANDOR by a margin of 0.01.

Table 1: Experimental results of RQ1. "*" denote CANDOR achieves significantly better results than the baselines.

	HumanEvalJava			Leetcode-Medium			Leetcode-Hard		
	Line	Branch	Mutation	Line	Branch	Mutation	Line	Branch	Mutation
LLM-Empirical	0.885	0.930	0.910 (2223/2443)	0.877	0.902	0.804 (491/611)	0.853	0.927	0.901 (774/859)
EvoSuite	0.961	0.942	0.858 (2096/2443)	0.959	**0.959**	0.845 (516/611)	0.984	0.976	0.888 (763/859)
CANDOR	**0.991***	**0.970**	**0.980*** (2384/2443)	**0.990***	0.949	**0.939*** (574/611)	**0.989***	**0.980**	**0.937*** (805/859)

For *line coverage*, CANDOR consistently achieves the highest scores across all datasets, with a minimum of 0.989 on the Leetcode-Hard dataset. Wilcoxon Signed Rank tests confirm that the differences between CANDOR and the strongest baseline, EvoSuite, are statistically significant. Similarly, for *mutation score*, CANDOR also performs best across the board, successfully killing 2384 (out of 2443), 574 (out of 611), and 805 (out of 859) mutants on HumanEvalJava, Leetcode-Medium, and Leetcode-Hard, respectively. Wilcoxon tests also confirms the significance of these improvements over EvoSuite. For *branch coverage*, although CANDOR achieves the best results on two out of three datasets, none of the differences between CANDOR and EvoSuite are statistically significant.

We also observe that CANDOR's performance remains stable as the dataset shifts from Leetcode-Medium to Leetcode-Hard, with only slight drops in line coverage (-0.01) and mutation score (-0.02), indicating robustness to increasing problem difficulty. However, since both datasets are still method-level and contain limited control-flow complexity, and further evaluation on project-level datasets is needed.

> Overall, CANDOR and EvoSuite produce comparable, high-quality test suites across all datasets. CANDOR achieves significantly better line coverage and mutation scores than both baselines across datasets with varying code complexity, while the differences in branch coverage between CANDOR and EvoSuite are not statistically significant.

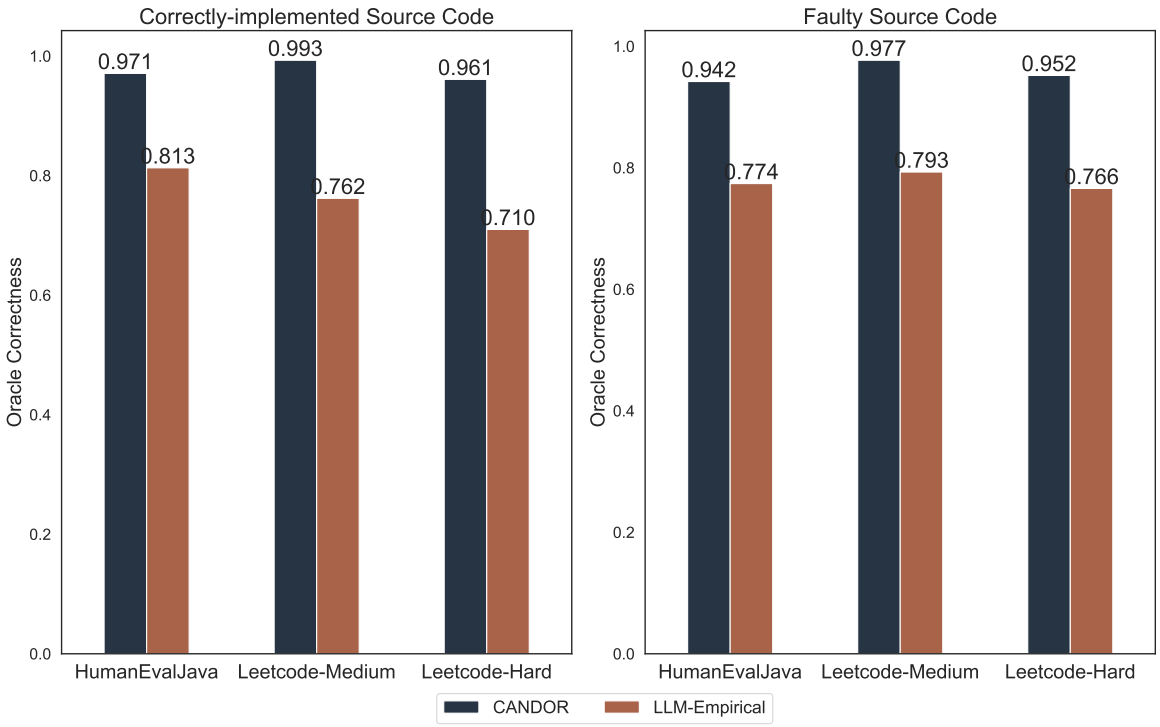

Figure 4: RQ2 results: Comparison of CANDOR and LLM-Empirical on correctly-implemented and faulty source code in terms of oracle correctness.

5.2 RQ2 Results: Test Oracle Quality

Figure 4 compares the correctness of oracles generated by CANDOR and LLM-Empirical. We exclude EvoSuite from this comparison since it can only produce regression oracles rather than specification-based oracles.

CANDOR achieves oracle correctness scores of 0.971, 0.993, and 0.961 on HumanEvalJava, Leetcode-Medium, and Leetcode-Hard, respectively. CANDOR substantially outperforms LLM-Empirical on all datasets, with the smallest margin of 0.158 ($0.971 - 0.834$) on HumanEvalJava and the largest margin of 0.251 ($0.961 - 0.710$) on Leetcode-Hard. Wilcoxon Signed Rank tests confirm all differences are statistically significant, and the effect size ($A_{12} = 0.875$) indicates a strong advantage.

To evaluate performance on faulty code, we also test on mutants generated by PiTest. Both methods show a decrease in oracle correctness, but CANDOR 's decline is smaller, widening the gap with LLM-Empirical. The gaps on HumanEvalJava, Leetcode-Medium and Leetcode-Hard are 0.168 ($0.942 - 0.774$), 0.184 ($0.977 - 0.793$) and 0.186 (0.952-0.766), respectively. This is likely because CANDOR leverages both source code and natural language descriptions for oracle generation, placing greater emphasis on the latter. As a result, errors in the source code have limited impact on CANDOR, since the description-derived requirements help correct the oracle. Wilcoxon tests again confirm significance of all the differences, with an even stronger effect size ($A_{12} = 0.960$).

> CANDOR generates significantly more accurate test oracles than LLM-Empirical across all datasets, and is robust to faulty code due to its incorporation of code descriptions.

5.3 RQ3 Results: Ablation study

Table 2 compares CANDOR with three ablation configurations: removing the *Planner* agent ("w/o Planner"), removing the *Requirement Engineer* agent ("w/o Req."), and removing the panel discussion ("w/o Panel").

Table 2: Experimental results of RQ3. "N/A" indicates current ablation configuration has no influence on this metric. "*" denote CANDOR achieves significantly better results than the baselines.

	HumanEvalJava				Leetcode-Medium				Leetcode-Hard			
	Line	Branch	Mutation	Oracle	Line	Branch	Mutation	Oracle	Line	Branch	Mutation	Oracle
CANDOR	**0.991***	**0.970***	**0.980*** (2384/2443)	**0.971***	**0.990***	**0.949***	**0.939*** (574/611)	**0.993**	**0.989***	**0.980***	**0.937*** (805/859)	**0.961***
w/o Planner	0.892	0.840	0.869 (2125/2443)	N/A	0.940	0.903	0.869 (531/611)	N/A	0.935	0.922	0.833 (716/859)	N/A
w/o Req.	N/A	N/A	N/A	0.939	N/A	N/A	N/A	0.988	N/A	N/A	N/A	0.920
w/o Panel	N/A	N/A	N/A	0.873	N/A	N/A	N/A	0.941	N/A	N/A	N/A	0.901

Removing the *Planner* significantly reduces test prefix quality across all datasets. The minimum decreases are observed on Leetcode-Medium with line coverage, branch coverage, and mutation score dropping by at least 0.050 ($0.990-0.940$), 0.046 ($0.949-0.903$), and 0.070 ($0.939-0.869$), respectively. Wilcoxon tests confirm these decreases are statistically significant. This underscores the critical role of the *Planner*, which analyzes current coverage and strategically plans to cover missing lines.

Removing the *Requirement Engineer* or the panel discussion primarily impacts oracle correctness. Without the *Requirement Engineer*, oracle correctness decreases slightly: 0.022 ($0.971-0.939$), 0.005 ($0.993-0.988$) and 0.041 ($0.961-0.920$) on HumanEvalJava, Leetcode-Medium, and Leetcode-Hard, respectively. Wilcoxon tests shows these changes on HumanEvalJava and Leetcode-Hard are significant while the decline on Leetcode-Medium is not. This likely suggests that the descriptions of source code in these three datasets are clear and well-structured, which reduces the need for explicit requirement parsing using the *Requirement Engineer* agent. However, in real-world scenarios with less clear documentation, this agent is expected to provide greater benefits.

In contrast, removing the panel discussion causes substantial and statistically significant drops in oracle correctness—0.098 ($0.971-0.873$), 0.052 ($0.993-0.941$), and 0.060 ($0.961-0.901$) across datasets. This aligns with expectations, as without the panel discussion, LLMs are more prone to uncertainty and hallucination, generating plausible but incorrect oracles more frequently.

Ablation results confirm the essential role of the *Planner* in achieving high-quality test prefixes, while the *Requirement Engineer* and panel discussion are effective for generating accurate test oracles.

6 Discussion

Despite the high effectiveness of incorporating LLMs for unit test generation, instructing LLMs to act as expected is a challenging task. In this section, we share several key insights obtained from our experiments about effective LLM instruction (Section 6.1), including the adoption of specialized agents, the strategy to mitigate LLM hallucination and the strategy to handle the verbosity of reasoning LLMs. We also report the limitations of our method in Section 6.2.

6.1 Key Insights

Simplifying Tasks via Specialized LLM Agents LLMs are increasingly studied not only due to their effectiveness but also their ease of use, as many tasks can be handled by crafting prompts for one single LLM Guo et al. [2024]. However, we found these single LLM-based approaches fall short in complex tasks like unit test generation. Unit test generation involves several sub-tasks, such as planning and generating test cases. When an LLM is asked to do both simultaneously, it often gets confused between roles. For example, we observed that when tasked with both planning and generating Java tests, the LLM sometimes produced syntax like `[1]` instead of the correct Java syntax `Arrays.asList(1)`. We hypothesize this happens due to the interference of the planning sub-task, which is often expressed in natural language or weak typed language like Python in real-world and LLMs' pretraining corpora. The LLM blends conventions of planning and Java test case generation, resulting in the syntactically wrong test case. By decomposing complex tasks and assigning each sub-task to a specialized LLM agent, we prevent this interference and improve the effectiveness of CANDOR in unit test generation.

Mitigating Hallucination via a Panel Discussion LLMs are notoriously prone to hallucination, generating plausible yet nonfactual content Huang et al. [2025]. We found that hallucination is particularly frequent in test oracle generations, where LLMs produce incorrect oracles even when clear instructions are present. For instance, when told to `square the`

`integer if its index is a multiple of 3`, an LLM may incorrectly leave the number unchanged. To address this, we use multiple LLMs to independently generate oracles and then compare their outputs. Since hallucinations are usually inconsistent, taking a consensus across models helps filter them out. Instead of simple majority voting, we let the LLMs explain their reasoning in a panel discussion, and determine the answer with the most consistent logic.

Stopping Overthinking via a Dual-LLM Pipeline Reasoning LLMs like DeepSeek R1 are effective but often overly verbose, producing lengthy outputs that significantly increase generation time. In our experiment, using DeepSeek directly for oracle evaluation sometimes resulted in outputs of over 10,000 tokens, taking hours to complete a single test file. In fact, we observed that correct oracles were often identified early by DeepSeek, but the model would continue second-guessing itself by thinking "`I think the correct oracle should be 147. But, wait, maybe I made a mistake ...`". To prevent this "overthinking", we truncate DeepSeek's output at 2,000 tokens and use a basic LLM to extract the correct oracle from its reasoning. This dual-LLM pipeline preserves the benefit of deep reasoning while keeping outputs concise.

6.2 Threats to validity

Construct Validity. We evaluate CANDOR using code coverage, mutation score, and oracle correctness. While real bug detection on datasets like Defects4J is another useful metric, we did not include it because such project-level datasets are beyond the scope of this work, as discussed earlier. Instead, we use mutation score as a proxy for bug-finding capability.

Internal Validity. Our results may be influenced by the choice of LLMs used in the framework. We use LLaMA 3.1 70B as the basic LLM and DeepSeek R1 as the reasoning LLM, selected based on preliminary experiments for their stability and open-source availability. While other popular models like GPT, Grok, and Mistral exist, exhaustively evaluating all LLMs is beyond the scope of this work. The choice of LLMs may affect performance, but we aim to demonstrate the potential effectiveness of our method rather than benchmark specific models, which might indeed yield even better results. Another potential threat is data leakage, wherein our datasets may exist in the pretraining data of the LLMs, leading to inflated performance. To mitigate this, we evaluate mutation score, which relies on detecting behavior changes in synthetically modified programs. These mutated versions are highly unlikely to appear in pretraining corpora, providing a more reliable evaluation of CANDOR.

Conclusion Validity. A potential threat is the randomness in LLM outputs, which may affect the consistency of results. To mitigate this, we repeat all experiments three times and apply statistical testing to assess the significance of the results across programs.

External Validity. One threat is that our evaluation is conducted on method-level datasets, and it is unclear how well CANDOR generalizes to more complex, project-level programs that involve dependencies. However, CANDOR is a multi-agent framework, which can be easily extended by introducing dedicated agents to handling dependencies. We leave this exploration for future work. Moreover, the strong performance of CANDOR on the LeetCodeJava dataset, which contains problems of varying difficulty, suggests that it is robust across diverse testing scenarios.

7 Related Work

Traditional methods rely on search-based and randomized techniques to generate test cases for Java programs Fraser and Arcuri [2011b], Rojas et al. [2017], McMinn [2004], Fraser and Arcuri [2011a], Pacheco and Ernst [2007]. Random testing tools, such as Randoop Pacheco and Ernst [2007], generate tests by executing sequences of randomly selected method calls. These approaches generate test inputs and regression oracles derived from the current implementation, which are effective for detecting behavioral changes across software versions but limited in validating functional correctness against intended specifications. This limitation arises because regression oracles assume the existing code is correct, potentially reinforcing bugs rather than detecting them. EvoSuite Fraser and Arcuri [2011a], which is the SOTA tool, uses genetic algorithms to systematically generate test suites for Java programs, focusing on maximizing code coverage.

To go beyond regression oracles, a few approaches have been proposed. TOGADinella et al. [2022] introduces a template-based approach for generating functional oracles by leveraging documentation and specification cues. More recently, TOGLLHossain and Dwyer [2024] combines EvoSuite-generated test prefixes with LLM-generated oracles through fine-tuned models. However, TOGLL depends on test prefixes generated by EvoSuite and requires extensive fine-tuning of large models, which can be resource-intensive and less adaptable to new domains. LLM-Empirical Siddiq et al. [2024] offers an alternative by using prompt-engineering to generate JUnit tests based on natural language descriptions, without fine-tuning. It focuses on generating both test prefixes and oracles using only

prompt engineering. CANDOR follows this line of research but introduces several key innovations, including the design of multiple specialized agents, the panel discussion to reduce hallucination and the dual-LLM pipeline of to extract structured information from the verbose output of reasoning LLMs. While recent advances in multi-agent LLM systems have achieved success in various software engineering domains (e.g. debugging Lee et al. [2024], fault localization Qin et al. [2024] and program repair Bouzenia et al. [2024]), their application to software testing remains underexplored. CANDOR is, to the best of our knowledge, is the first multi-agent, end-to-end JUnit test generator, laying the groundwork for future research in agent-based automated testing.

8 Conclusion

In this work, we presented CANDOR, a novel multi-agent, end-to-end framework for generating high-quality unit tests with accurate oracles. CANDOR leverages a panel discussion to mitigate hallucinations and employs a dual-LLM pipeline to reduce overthinking during reasoning.

Our extensive evaluation on HumanEvalJava and LeetCode demonstrates that CANDOR achieves better performance than EvoSuite in terms of line coverage and mutation score, while producing accurate specification-based oracles, which EvoSuite cannot. Ablation studies highlight the critical roles of key agents, including the *Planner* for enhancing test prefix quality and the *Requirement Engineer* along with the panel discussion for improving oracle correctness. For future work, we plan to explore project-level test generation and investigate more advanced variants of panel discussions to further reduce hallucination.

Acknowledgment

This work was supported by a research grant from Huawei Technologies and Research Ireland grant 13/RC/209. Lionel C. Briand's contribution was partially funded by the Research Chair and Discovery Grant programs of the Natural Sciences and Engineering Research Council of Canada (NSERC).

References

Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius Carvalho Lopes. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, pages 313–322, 2024.

Kent Beck. *Test driven development: By example*. Addison-Wesley Professional, 2022.

Ermira Daka, José Campos, Gordon Fraser, Jonathan Dorn, and Westley Weimer. Modeling readability to improve unit tests. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 107–118, 2015.

Ermira Daka and Gordon Fraser. A survey on unit testing practices and problems. In *2014 IEEE 25th International Symposium on Software Reliability Engineering*, pages 201–211. IEEE, 2014.

Danielle Gonzalez, Joanna CS Santos, Andrew Popovich, Mehdi Mirakhorli, and Mei Nagappan. A large-scale study on the usage of testing patterns that address maintainability attributes: patterns for ease of modification, diagnoses, and comprehension. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pages 391–401. IEEE, 2017.

Gordon Fraser and Andrea Arcuri. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, pages 416–419, 2011a.

Carlos Pacheco, Shuvendu K Lahiri, Michael D Ernst, and Thomas Ball. Feedback-directed random test generation. In *29th International Conference on Software Engineering (ICSE'07)*, pages 75–84. IEEE, 2007.

Sina Shamshiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 201–211. IEEE, 2015.

Soneya Binta Hossain and Matthew Dwyer. Togll: Correct and strong test oracle generation with llms. *arXiv preprint arXiv:2405.03786*, 2024.

Jonathan G Tullis and Aaron S Benjamin. On the effectiveness of self-paced learning. *Journal of memory and language*, 64(2):109–118, 2011.

Domenico Serra, Giovanni Grano, Fabio Palomba, Filomena Ferrucci, Harald C Gall, and Alberto Bacchelli. On the effectiveness of manual and automatic unit test generation: ten years later. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, pages 121–125. IEEE, 2019. ISBN 1728134129.

Barton P Miller, Lars Fredriksen, and Bryan So. An empirical study of the reliability of unix utilities. *Communications of the ACM*, 33(12):32–44, 1990.

Andrea Fioraldi, Alessandro Mantovani, Dominik Maier, and Davide Balzarotti. Dissecting american fuzzy lop: a fuzzbench evaluation. *ACM transactions on software engineering and methodology*, 32(2):1–26, 2023.

Christoph Csallner and Yannis Smaragdakis. Jcrasher: an automatic robustness tester for java. *Software: Practice and Experience*, 34(11):1025–1050, 2004.

Carlos Pacheco and Michael D Ernst. Randoop: feedback-directed random testing for java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, pages 815–816, 2007.

Carlos Pacheco, Shuvendu K Lahiri, and Thomas Ball. Finding errors in. net with feedback-directed random testing. In *Proceedings of the 2008 international symposium on Software testing and analysis*, pages 87–96, 2008.

Marija Selakovic, Michael Pradel, Rezwana Karim, and Frank Tip. Test generation for higher-order functions in dynamic languages. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–27, 2018.

Ellen Arteca, Sebastian Harner, Michael Pradel, and Frank Tip. Nessie: Automatically testing javascript apis with asynchronous callbacks. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1494–1505, 2022.

Patrice Godefroid, Nils Klarlund, and Koushik Sen. Dart: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*, pages 213–223, 2005.

Koushik Sen, Darko Marinov, and Gul Agha. Cute: A concolic unit testing engine for c. *ACM SIGSOFT software engineering notes*, 30(5):263–272, 2005.

Cristian Cadar, Vijay Ganesh, Peter M Pawlowski, David L Dill, and Dawson R Engler. Exe: Automatically generating inputs of death. *ACM Transactions on Information and System Security (TISSEC)*, 12(2):1–38, 2008.

Nikolai Tillmann, Jonathan De Halleux, and Tao Xie. Transferring an automated test generation tool to practice: From pex to fakes and code digger. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*, pages 385–396, 2014.

Gordon Fraser and Andrea Arcuri. Evolutionary generation of whole test suites. In *2011 11th International Conference on Quality Software*, pages 31–40. IEEE, 2011b.

Gordon Fraser, Matt Staats, Phil McMinn, Andrea Arcuri, and Frank Padberg. Does automated unit test generation really help software testers? a controlled empirical study. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 24(4):1–49, 2015.

Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*, pages 2130–2141, 2022.

Facundo Molina, Alessandra Gorla, and Marcelo d'Amorim. Test oracle automation in the era of llms. *ACM Transactions on Software Engineering and Methodology*, 2024.

Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2023.

Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 919–931. IEEE, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.

Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868*, 2022.

Leetcodejava dataset, Accessed: 2025a. URL `https://leetcode.com`.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Andrew Ettles, Andrew Luxton-Reilly, and Paul Denny. Common logic errors made by novice programmers. In *Proceedings of the 20th Australasian Computing Education Conference*, pages 83–89, 2018.

Pitest official website, Accessed: 2025b. URL `https://pitest.org/`.

René Just, Darioush Jalali, and Michael D. Ernst. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ISSTA 2014, page 437–440, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450326452. doi:10.1145/2610384.2628055. URL `https://doi.org/10.1145/2610384.2628055`.

Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020.

Hadi Hemmati. How effective are code coverage criteria? In *2015 IEEE International Conference on Software Quality, Reliability and Security*, pages 151–156. IEEE, 2015.

Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd international conference on software engineering*, pages 1–10, 2011.

Langchain official website, Accessed: 2025c. URL `https://www.langchain.com/`.

Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

José Miguel Rojas, Mattia Vivanti, Andrea Arcuri, and Gordon Fraser. A detailed investigation of the effectiveness of whole test suite generation. *Empirical Software Engineering*, 22:852–893, 2017.

Phil McMinn. Search-based software test data generation: a survey. *Software testing, Verification and reliability*, 14(2): 105–156, 2004.

Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jen-tse Huang, Zhouruixin Zhu, Lingming Zhang, and Michael R Lyu. A unified debugging approach via llm-based multi-agent synergy. *arXiv preprint arXiv:2404.17153*, 2024.

Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. Agentfl: Scaling llm-based fault localization to project-level context. *arXiv preprint arXiv:2403.16362*, 2024.

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134*, 2024.